TEAM VEGAVATH

Official Website — Architecture Document

v1.0 — Phase 1 Build Reference

March 2026

1. Project Overview

This document defines the complete architecture for the Vegavath Racing official website rebuild. It covers the route map, component tree, database schema, storage structure, environment variables, and build phases. No code is written until this document is reviewed and approved.

1.1 Guiding Principles1.1 Guiding Principles

- Zero media files in the git repository — everything goes to Cloudflare R2
- No hardcoded content that requires a code deployment to change
- Admin controls all dynamic content: events, team members, recruitments, gallery, sponsors
- Clean TypeScript — no any, no @ts-ignore, no ignoreBuildErrors
- Mobile-first — primary audience is college students on phones
- Zero-cost deployment for Phase 1 — must run fully on free tiers with no credit card required
- Architecture must scale without refactor when paid resources become available
- Junior-maintainable — any new team member can understand the codebase

1.2 Stack — Final Locked Decisions1.2 Stack — Final Locked Decisions

Layer	Choice	Free Tier Limit	Why
Framework	Next.js 15 App Router + TypeScript	Unlimited	App Router handles dynamic routes cleanly
Styling	Tailwind CSS v3	Free, MIT	Design tokens ready for when colour scheme approved
Animations	Framer Motion	Free, MIT	Pairs cleanly with R3F; UI transitions only
3D Model	React Three Fiber + Drei	Free, MIT	Kart model below hero section
Database	Neon (Postgres)	0.5GB, 190hrs compute/mo	India-accessible, portable SQL
Media / CDN	Cloudflare R2	10GB, zero egress fees	All images, videos, logos stored here
Auth (Admin)	NextAuth.js v5	Free, MIT	Session-based, Next.js native
Forms (Phase 2)	n8n (self-hosted)	Free, self-hosted	Event registration automation
Hosting	Vercel (decide later)	Hobby plan free	Native Next.js, build-agnostic codebase

2. Route Map

Complete list of all routes, their rendering strategy, and data sources.

Route	Page	Strategy	Revalidate	Data Source
/	Home	ISR	60s	Neon DB (events preview, sponsors strip)
/about	About	ISR	120s	Static content + Neon (timeline, sponsors carousel)
/events	Events Index	ISR	60s	Neon DB (all events, filtered)
/events/[slug]	Event Detail	SSR	none	Neon DB — registration status must be live
/gallery	Gallery	ISR	120s	Neon DB (gallery items + R2 URLs)
/crew	The Crew	ISR	120s	Neon DB (team members by tier)
/sponsors	Sponsors	ISR	120s	Neon DB (sponsors list)
/join	Join Us	SSR	none	Neon DB — recruitment flag must be live
/admin	Admin Login	Client	none	NextAuth session check
/admin/*	Admin Pages	SSR + Protected	none	Neon DB — always fresh
*	404 Page	Static	none	None

2.1 Rendering Strategy Rationale2.1 Rendering Strategy Rationale

ISR (Incremental Static Regeneration) is used for all public read-heavy pages. Pages are statically generated and served from CDN edge, revalidating in the background every 60-120 seconds. This means a team member added via admin appears on /crew within 2 minutes without a full redeploy — and the page serves instantly from cache during traffic spikes.

SSR is reserved for routes where stale data causes real problems: /join must always reflect the live recruitment_open flag, and /events/[slug] must always show the live registration status so students don't see an open form when registration is actually closed.

2.2 Admin Route Protection2.2 Admin Route Protection

All /admin/* routes (except /admin login) are protected by NextAuth middleware. Unauthenticated requests redirect to /admin. Session stored server-side. No client-side auth checks — middleware handles everything at the edge. All admin API routes validate session server-side before any DB operation regardless of middleware.

3. Folder Structure

Clean separation of concerns. No business logic in components. No media in the repo.

```
vegavath/
├── src/
│   ├── app/                          # Next.js App Router
│   │   ├── (public)/                 # Public route group
│   │   │   ├── page.tsx              # /
│   │   │   ├── about/page.tsx        # /about
│   │   │   ├── events/               # /events
│   │   │   │   ├── page.tsx
│   │   │   │   └── [slug]/page.tsx   # /events/[slug]
│   │   │   ├── gallery/page.tsx      # /gallery
│   │   │   ├── crew/page.tsx         # /crew
│   │   │   ├── sponsors/page.tsx     # /sponsors
│   │   │   └── join/page.tsx         # /join
│   │   ├── (admin)/                  # Admin route group
│   │   │   ├── admin/page.tsx        # /admin login
│   │   │   └── admin/
│   │   │       ├── dashboard/page.tsx
│   │   │       ├── events/page.tsx
│   │   │       ├── team/page.tsx
│   │   │       ├── gallery/page.tsx
│   │   │       ├── sponsors/page.tsx
│   │   │       └── settings/page.tsx
│   │   ├── api/                      # API routes
│   │   │   ├── auth/[...nextauth]/route.ts
│   │   │   ├── events/route.ts
│   │   │   ├── team/route.ts
│   │   │   ├── gallery/route.ts
│   │   │   ├── sponsors/route.ts
│   │   │   ├── join/route.ts
│   │   │   └── admin/
│   │   │       ├── events/route.ts
│   │   │       ├── team/route.ts
│   │   │       ├── gallery/route.ts
│   │   │       ├── sponsors/route.ts
│   │   │       └── settings/route.ts
│   │   ├── layout.tsx                # Root layout
│   │   ├── not-found.tsx             # 404
│   │   └── globals.css
│   ├── components/
│   │   ├── layout/                   # Nav, Footer, Cursor
│   │   ├── home/                     # Home page sections
│   │   ├── about/                    # About page sections
│   │   ├── events/                   # Event cards, filters
│   │   ├── gallery/                  # Masonry, lightbox
│   │   ├── crew/                     # Member cards
│   │   ├── sponsors/                 # Marquee, sponsor cards
│   │   ├── join/                     # Application form
│   │   ├── admin/                    # Admin UI panels
│   │   └── ui/                       # Shared: Button, Modal, Skeleton, Toast
│   ├── lib/
│   │   ├── db.ts                     # Neon connection
│   │   ├── r2.ts                     # Cloudflare R2 client
│   │   ├── auth.ts                   # NextAuth config
│   │   └── utils.ts                  # Shared utilities
│   ├── types/                        # TypeScript interfaces
```

```
│   │   ├── event.ts
│   │   ├── member.ts
│   │   ├── gallery.ts
│   │   ├── sponsor.ts
│   │   └── settings.ts
│   └── middleware.ts                # Admin route protection
├── public/
│   ├── icons/                       # Only SVG/PNG icons
│   └── models/                      # .glb kart model file
├── .env.local                       # Never committed
├── next.config.ts
├── tailwind.config.ts
└── tsconfig.json
```

4. Database Schema (Neon / Postgres)

All tables use UUID primary keys. All timestamps are UTC. RLS is not used — auth is handled at the API layer via NextAuth middleware.

4.1 events**4.1 events**

Column	Type	Notes
id	UUID PRIMARY KEY	default gen_random_uuid()
slug	TEXT UNIQUE NOT NULL	e.g. ignition-1, embedx-2
title	TEXT NOT NULL	Display name
category	TEXT NOT NULL	workshops | competitions | talks | other
status	TEXT NOT NULL	upcoming | past
description	TEXT	Event description paragraph
event_date	DATE NOT NULL	
logo_url	TEXT	R2 URL for event logo
cover_image_url	TEXT	R2 URL for card thumbnail
registration_open	BOOLEAN DEFAULT false	Admin toggles this
registration_form_url	TEXT	External form or internal route
sponsors	JSONB	Array of sponsor IDs for this event
created_at	TIMESTAMPTZ DEFAULT now()	
updated_at	TIMESTAMPTZ DEFAULT now()	

4.2 team_members**4.2 team_members**

Column	Type	Notes
id	UUID PRIMARY KEY	
name	TEXT NOT NULL	
role	TEXT NOT NULL	e.g. Club Head, Design Head, Member
tier	TEXT NOT NULL	core | crew | legacy
domain	TEXT	Automotive | Robotics | Design | Media | Marketing | Programming
quote	TEXT	Personal quote shown on card
photo_url	TEXT	R2 URL
display_order	INTEGER DEFAULT 0	Controls card ordering within tier
is_active	BOOLEAN DEFAULT true	Set false to hide without deleting
created_at	TIMESTAMPTZ DEFAULT now()	

4.3 gallery_items**4.3 gallery_items**

Column	Type	Notes
id	UUID PRIMARY KEY	
event_id	UUID REFERENCES events(id)	Links to event for filtering
event_label	TEXT NOT NULL	Display label e.g. Ignition 1.0
type	TEXT NOT NULL	image | video
url	TEXT NOT NULL	R2 URL for image; YouTube embed URL for video
caption	TEXT	Shown on hover and lightbox
taken_at	DATE	Photo date for ordering
display_order	INTEGER DEFAULT 0	
created_at	TIMESTAMPTZ DEFAULT now()	

4.4 sponsors**4.4 sponsors**

Column	Type	Notes
id	UUID PRIMARY KEY	
name	TEXT NOT NULL	
logo_url	TEXT NOT NULL	R2 URL
website_url	TEXT	Optional link on logo click
description	TEXT	Shown in About carousel and /sponsors page
tier	TEXT	title | gold | silver | community
is_active	BOOLEAN DEFAULT true	Toggle visibility without deleting
display_order	INTEGER DEFAULT 0	
created_at	TIMESTAMPTZ DEFAULT now()	

4.5 applications (Join Us)4.5 applications (Join Us)

Column	Type	Notes
id	UUID PRIMARY KEY	
name	TEXT NOT NULL	
email	TEXT NOT NULL	
domain_interest	TEXT NOT NULL	Automotive | Robotics | Design | Media | Marketing
portfolio_url	TEXT	Optional
status	TEXT DEFAULT 'pending'	pending | reviewed | accepted | rejected
submitted_at	TIMESTAMPTZ DEFAULT now()	

4.6 site_settings4.6 site_settings

Column	Type	Notes
key	TEXT PRIMARY KEY	
value	TEXT NOT NULL	
updated_at	TIMESTAMPTZ DEFAULT now()	

Key-value store for admin-controlled flags. Keys include:

- recruitment_open — true | false
- maintenance_mode — true | false
- maintenance_message — display string
- contact_email — club email address
- contact_phone — club phone number
- contact_address — college address string
- instagram_url, linkedin_url, github_url — social links

4.7 Required Database Indexes4.7 Required Database Indexes

All indexes defined at table creation time. These are not optional — missing indexes on filtered/sorted columns cause full table scans which will exhaust Neon free compute on burst traffic.

Table	Column(s)	Index Type	Reason
events	slug	UNIQUE (already PK constraint)	Slug lookups for /events/[slug]
events	status	B-tree	Filter upcoming vs past on /events and home preview
events	event_date	B-tree	Sort events reverse chronologically
team_members	tier	B-tree	Filter core / crew / legacy on /crew
team_members	display_order	B-tree	Sort within tier
team_members	is_active	B-tree	Filter hidden members
gallery_items	event_id	B-tree	Filter gallery by event on /gallery
gallery_items	display_order	B-tree	Sort within event gallery
sponsors	is_active	B-tree	Filter hidden sponsors
sponsors	display_order	B-tree	Sort sponsor display order

Table	Column(s)	Index Type	Reason
applications	submitted_at	B-tree	Sort applications by date in admin

Create indexes in the same migration file as the tables. Example:

```
CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_events_date ON events(event_date DESC);
CREATE INDEX idx_gallery_event ON gallery_items(event_id);
CREATE INDEX idx_members_tier ON team_members(tier, display_order);
```

5. Component Architecture

Components are organised by feature. Shared primitives live in /ui. No page-level business logic — pages are thin wrappers that fetch data and pass it down.

5.0 Services Layer (Non-Negotiable Rule)5.0 Services Layer (Non-Negotiable Rule)

All database query logic lives in src/lib/services/. API routes and server components call service functions — they never write raw SQL themselves. This prevents the copy-paste logic drift that destroyed the old codebase. If both /api/events and /api/admin/events need to fetch events, they call the same getEvents() function.

Service File	Exports	Called By
services/events.ts	getEvents(), getEventBySlug(), createEvent(), updateEvent(), archiveEvent()	public + admin API routes, page server components
services/team.ts	getMembers(), getMembersByTier(), createMember(), updateMember(), toggleActive()	public + admin API routes, /crew page
services/gallery.ts	getGalleryItems(), getGalleryByEvent(), createGalleryItem(), deleteGalleryItem()	public + admin API routes, /gallery page
services/sponsors.ts	getSponsors(), getActiveSponsors(), createSponsor(), updateSponsor()	public + admin API routes, all pages with sponsors
services/applications.ts	createApplication(), getApplications(), updateApplicationStatus()	/api/join, /api/admin/settings
services/settings.ts	getSetting(), setSetting(), getAllSettings()	/api/admin/settings, root layout (footer)

5.1 Layout Components5.1 Layout Components

Component	Props	Notes
<Navbar />	none	Sticky, logo + nav links. Active state from usePathname.
<Footer />	none	4-col layout. Social links from site_settings via server fetch.
<RacingCursor />	none	Desktop only. Detects touch, disables itself. Togglable via localStorage.
<CursorToggle />	none	Bottom-right toggle button. Persists preference.
<PageTransition />	children	Framer Motion route transition wrapper.

5.2 Home Page Components5.2 Home Page Components

Component	Props	Notes
<HeroSection />	none	Title, tagline, Join Us CTA button.
<KartModelSection />	none	Full-width R3F canvas. .glb file from /public/models/. Drag to rotate planned Phase 2.
<DomainsSection />	domains: Domain[]	5-card grid. Data is static — domains don't change.
<EventsPreviewSection />	events: Event[]	Shows 3 upcoming + 3 past. Single View All Events CTA.
<SponsorsStrip />	sponsors: Sponsor[]	Static 6-logo row. Different from marquee carousel.
<JoinCTASection />	none	Join Now button → /join.

5.3 Shared / Reusable Components5.3 Shared / Reusable Components

Component	Props	Notes
<EventCard />	event: Event, size?: 'sm'|'lg'	Used on home preview, /events grid, /events/[slug] related.
<MemberCard />	member: TeamMember	Photo + name + role + quote. Hover sliding lines animation.
<SponsorCard />	sponsor: Sponsor	Logo + name + description. Used in carousel and /sponsors grid.
<SponsorMarquee />	sponsors: Sponsor[]	CSS infinite scroll marquee. Used on /about.
<GalleryMasonry />	items: GalleryItem[]	3-col masonry. Handles both image and video cells.

Component	Props	Notes
<GalleryLightbox />	item: GalleryItem, onClose: fn	Full-screen overlay. Image zoom or YT embed.
<TimelineSection />	entries: TimelineEntry[]	Alternating left/right cards. Centre vertical line.
<RecruitmentForm />	isOpen: boolean	Controlled form. Shows closed state if isOpen=false.
<AdminProtectedLayout />	children	Wraps all /admin/* routes. Checks session server-side.
<SkeletonCard />	variant: 'event'|'member'|'gallery'	Loading skeleton per card type.

6. Cloudflare R2 Storage Structure

One R2 bucket: vegavath-media. Organised by content type. All URLs are public read via R2 public domain or custom domain if configured.

```
vegavath-media/
├── events/
│   ├── ignition-1/
│   │   ├── logo.png
│   │   ├── cover.jpg
│   │   └── gallery/
│   │       ├── img-001.jpg
│   │       └── img-002.jpg
│   └── embedx-2/
│       ├── logo.png
│       ├── cover.jpg
│       └── gallery/
├── team/
│   ├── core/
│   │   ├── naveen-s.jpg
│   │   └── ankush-gowda.jpg
│   ├── crew/
│   └── legacy/
├── sponsors/
│   ├── solidworks.svg
│   ├── xylem.png
│   ├── ather-energy.png
│   ├── mahindra.png
│   └── bmw-motorrad.svg
└── site/
    ├── logo.png
    └── og-image.jpg
```

Naming convention: lowercase, hyphenated, no spaces. Matches slug pattern used in DB. When admin adds a new event 'Bootcamp 2026', slug = bootcamp-2026, R2 path = events/bootcamp-2026/.

7. Environment Variables

All secrets in .env.local. Never committed. Rotate immediately if exposed.

Variable	Used In	Description
DATABASE_URL	src/lib/db.ts	Neon Postgres connection string
NEXTAUTH_SECRET	src/lib/auth.ts	Random 32-char string. Generate: openssl rand -base64 32
NEXTAUTH_URL	src/lib/auth.ts	http://localhost:3000 (dev) / production URL
ADMIN_USERNAME	src/lib/auth.ts	Admin login username
ADMIN_PASSWORD_HASH	src/lib/auth.ts	bcrypt hash of password. Never store plaintext.
R2_ACCOUNT_ID	src/lib/r2.ts	Cloudflare account ID
R2_ACCESS_KEY_ID	src/lib/r2.ts	R2 API access key
R2_SECRET_ACCESS_KEY	src/lib/r2.ts	R2 API secret
R2_BUCKET_NAME	src/lib/r2.ts	vegavath-media
R2_PUBLIC_URL	src/lib/r2.ts	Public base URL for R2 objects
NEXT_PUBLIC_R2_PUBLIC_URL	Client components	Same as above, exposed to browser for image src

7.1 What We Removed vs Old Site**7.1 What We Removed vs Old Site**

- NEXT_PUBLIC_SUPABASE_URL — replaced by DATABASE_URL (Neon)
- NEXT_PUBLIC_SUPABASE_ANON_KEY — removed entirely
- SUPABASE_SERVICE_ROLE_KEY — removed entirely
- ADMIN_PASSWORD — replaced by ADMIN_PASSWORD_HASH (no plaintext)
- ADMIN_COOKIE_SECRET — replaced by NEXTAUTH_SECRET
- NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME — replaced by R2
- GOOGLE_SERVICE_ACCOUNT — Phase 2 only if Google Sheets export needed

8. Admin Panel Capabilities

The admin panel is accessible at /admin. Login with username + password (bcrypt hashed). Session persists for 24 hours. All activities should be logged to the applications table or a separate audit log.

8.1 Events Management (/admin/events)8.1 Events Management (/admin/events)

- Create new event → auto-generates slug → creates R2 folder path
- Set title, date, category, description, status (upcoming/past)
- Upload event logo and cover image → stored to R2
- Toggle registration_open flag → /join shows open or closed state
- Assign sponsors to event (multi-select from sponsors table)
- Edit or delete existing events

8.2 Team Management (/admin/team)8.2 Team Management (/admin/team)

- Add member: name, role, tier (core/crew/legacy), domain, quote, photo upload → R2
- Reorder members within tier via display_order
- Toggle is_active to hide without deleting
- Edit or delete members

8.3 Gallery Management (/admin/gallery)8.3 Gallery Management (/admin/gallery)

- Upload images per event → stored to R2 under events/[slug]/gallery/
- Add YouTube embed URLs for video entries
- Set caption for each item
- Delete items (removes DB record + R2 object)

8.4 Sponsors Management (/admin/sponsors)8.4 Sponsors Management (/admin/sponsors)

- Add sponsor: name, logo upload → R2, website URL, description, tier
- Toggle is_active
- Reorder via display_order

8.5 Site Settings (/admin/settings)8.5 Site Settings (/admin/settings)

- Toggle recruitment_open — controls /join form visibility
- Toggle maintenance_mode — shields public pages
- Edit contact info (email, phone, address)
- Edit social media URLs (Instagram, LinkedIn, GitHub)
- View recent join applications with status management

8.6 Join Form — Validation & Spam Rules8.6 Join Form — Validation & Spam Rules

The /join application form is the only public form that writes to the DB. It must be hardened at the API level.

• Server-side validation on every field before DB insert — never trust client
• name: required, min 2 chars, max 100 chars, no HTML
• email: required, valid email format (regex check server-side)
• domain_interest: required, must match exactly one of the defined domain enum values
• portfolio_url: optional, if present must start with https://
• Honeypot field: hidden input named 'website' — if it has any value, silently reject the submission (bots fill it, humans don't)
• Rate limiting: max 3 submissions per IP per 60 minutes — tracked in memory or Vercel Edge middleware
• On validation failure: return specific error per field, not a generic 'invalid' message

Honeypot implementation — add to form HTML, hide via CSS (not display:none which bots detect):

```
// In form JSX
<input name='website' className='sr-only' tabIndex={-1} autoComplete='off' />
// In API route — reject silently if filled
if (body.website) return NextResponse.json({ success: true }); // fake success
```

9. Build Phases

Phase 1 — Core Site (Current Build)**Phase 1 — Core Site (Current Build)**

Everything needed to replace the old site with zero regressions.

Area	Deliverable
Scaffold	Fresh Next.js 15 + TypeScript + Tailwind, strict config, ESLint, GitHub Actions CI
DB	Neon setup, all tables + indexes created, seed data for existing events/team
R2	Bucket created, existing media migrated from repo to R2 via migration script
Auth	NextAuth admin login, middleware protection
Layout	Navbar, Footer, RacingCursor, CursorToggle, PageTransition
Home (/)	Hero, KartModel section (placeholder), Domains, Events preview, Sponsors strip, Join CTA
About (/about)	Who We Are, Mission, What We Do, Sponsor carousel (marquee), Journey+Vision, Timeline, Values
Events (/events)	Filterable grid, all past events from DB, LIMIT 20 default
Event Detail (/events/[slug])	Dynamic route, event data from DB, gallery from R2
Gallery (/gallery)	Masonry grid, event filters (dynamic from DB), lightbox, YT embeds, LIMIT 30 default
Crew (/crew)	Three-tier member grid, photo from R2, Join CTA strip
Sponsors (/sponsors)	Full logo grid from DB, Become a Sponsor CTA
Join (/join)	Recruitment open/closed (DB-driven), domain cards, application form with honeypot + validation
Admin	Login, dashboard, events CRUD, team CRUD, gallery upload, sponsors CRUD, settings
404	Custom branded not-found page
Security	No plaintext secrets, no exposed endpoints, proper auth on all admin routes

Phase 1 Pre-Launch — Migration Plan**Phase 1 Pre-Launch — Migration Plan**

Before going live, all existing media must be moved from the old repo to R2. This is a scripted process, not a manual one.

Step	Action	Tool
1	Clone old repo locally to a temp folder outside the new project	git clone
2	Audit all files in public/assets/gallery/ and src/lib/crewimg/	PowerShell: Get-ChildItem -Recurse
3	Run upload script: iterate all files, upload to correct R2 path per naming convention	Node.js script using @aws-sdk/client-s3
4	Run seed script: insert DB records for all events, team members, gallery items with R2 URLs	Node.js script using @neondatabase/serverless
5	Verify: spot-check 10 random images load correctly from R2 URLs in browser	Manual
6	Verify: all pages render correctly with DB data (no broken images)	Manual + CI
7	Switch DNS / update Netlify → Vercel	Hosting platform
8	Confirm old repo media can be removed from git history	git filter-repo (optional)

The upload and seed scripts live in /scripts/migrate/ and are committed to the repo. They are run once and then kept for reference. They must be idempotent — safe to run multiple times without creating duplicate records.

Phase 2 — Enhancements (Post Phase 1)**Phase 2 — Enhancements (Post Phase 1)**

- n8n integration for event registration form automation
- Kart model drag-to-rotate interaction
- Google Sheets export for applications
- Image optimisation pipeline (auto-WebP conversion on R2 upload)
- Timeline data DB-driven (currently static placeholder)
- Email notifications on new join applications

10. Development Rules

These rules are non-negotiable. Any PR that violates them gets rejected.

ALWAYSALWAYS

- TypeScript strict mode. Every prop typed. No implicit any.
- Server Components for data fetching. Client Components only when interactivity requires it.
- Images via next/image with R2 URLs. Never <img> tags.
- Media uploaded to R2 via admin panel. Never committed to repo.
- Environment variables for all secrets. Never hardcoded.
- Error boundaries on all async data fetches.
- Loading states (skeleton cards) for all DB-fetched sections.
- Mobile-first CSS — design for 375px first, scale up.
- All list queries must use LIMIT — gallery default 30, events default 20, never unbounded SELECT *
- All DB logic goes in src/lib/services/ — API routes call service functions, never raw SQL in routes.

NEVERNEVER

- ignoreBuildErrors or ignoreDuringBuilds in next.config.ts.
- Binary files (images, videos, PDFs) committed to git.
- Plaintext passwords or secrets anywhere in code or commits.
- Admin API routes without session verification.
- force-dynamic on root layout — use it per-route only where needed.
- Unprotected admin UI routes.
- Direct DOM manipulation — React handles the DOM.
- Inline styles — use Tailwind classes.

11. First Steps — Scaffold Commands

Run these in order in your new folder. Windows PowerShell compatible.

Step 1: Create project

```
npx create-next-app@latest vegavath --typescript --tailwind --eslint --app --src-dir --import-alias '@/*'
```

Step 2: Install dependencies

```
cd vegavath
npm install framer-motion @react-three/fiber @react-three/drei three
npm install next-auth@beta @auth/core
npm install @neondatabase/serverless
npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
npm install bcryptjs
npm install -D @types/bcryptjs @types/three
```

Step 3: Strict TypeScript config

In tsconfig.json, ensure: strict: true, noUncheckedIndexedAccess: true

Step 4: Clean next.config.ts

No ignoreBuildErrors. No ignoreDuringBuilds. Add R2 public URL to images.remotePatterns.

Step 5: Create .env.local

Add all variables from Section 7. Generate NEXTAUTH_SECRET with: openssl rand -base64 32

After scaffold is confirmed working, report back. Architecture phase complete — build phase begins.

12. Production Readiness

This section defines real-world stress handling. The site must survive event registration bursts, gallery browsing spikes, and admin actions without degrading public-facing performance.

12.1 Expected Traffic Reality12.1 Expected Traffic Reality

Realistic load patterns for a college club site at PESU ECC:

- Event registration opening → sudden burst, estimated 50–200 concurrent users in first 10 minutes
- Gallery browsing → sustained bandwidth load, many parallel image requests
- Normal traffic → low and consistent, 5–20 concurrent

ISR handles the gallery and general browsing load by serving cached pages from CDN. The only routes that hit Neon during a burst are /join and /events/[slug] — both are lightweight single-row queries.

12.2 Database Load Protection12.2 Database Load Protection

- ISR pages never hit Neon on every request — only on revalidation cycles
- No N+1 queries — fetch all required data in one query per page, not one query per item
- No heavy joins — denormalise where needed (e.g. sponsor names stored in event JSONB column)
- Neon connection via @neondatabase/serverless uses HTTP pooling — safe for serverless environments
- If Neon is ever swapped out, it is a one-line DATABASE_URL change — standard Postgres wire protocol

12.3 Media Delivery — Thumbnail Strategy12.3 Media Delivery — Thumbnail Strategy

Gallery grid and event cards must never load full-resolution images. Two size tiers per image:

Context	Size	Format	Naming Convention
Card thumbnails, gallery grid	800px wide max	WebP	img-001-thumb.webp
Lightbox, full view	Original resolution	JPG/PNG preserved	img-001.jpg
Member photos (crew page)	600px wide max	WebP	naveen-s-thumb.webp
Sponsor logos	Original SVG preferred	SVG or WebP	solidworks.svg
Event covers (cards)	800px wide max	WebP	cover-thumb.webp

On admin upload: store original to R2, generate compressed WebP thumbnail server-side using sharp (already in Next.js pipeline), store thumbnail alongside original. DB stores both URLs.

12.4 R2 Cache Headers12.4 R2 Cache Headers

R2 objects are immutable — filenames include content identity (slug-based). Set long-lived Cache-Control headers on all R2 objects at upload time:

```
Cache-Control: public, max-age=31536000, immutable
```

This means browsers and CDN cache media for 1 year. When an image is replaced (e.g. team member photo update), upload under a new filename and update the DB URL. Never overwrite existing R2 objects.

12.5 Error Handling Requirements12.5 Error Handling Requirements

Every async server component that fetches from Neon must have an error boundary. Failures must show a degraded UI, not a crash.

Component	Failure State
<EventsPreviewSection />	Show 'Events coming soon' placeholder
<GalleryMasonry />	Show 'Gallery temporarily unavailable'
<SponsorMarquee />	Hide section silently — not critical
<CrewGrid />	Show 'Team info unavailable'
<RecruitmentForm />	Show 'Form unavailable, contact us directly'
Admin data tables	Show error message with retry button

Server errors must be logged. Minimum: console.error with route context. Future: structured logging to an audit table in Neon.

12.6 Admin API Security12.6 Admin API Security

- Every /api/admin/* route validates NextAuth session as first operation — before any DB read or write
- All input validated server-side — never trust client data
- Rate limiting on /api/join (application submissions) — max 3 submissions per IP per hour via simple in-memory counter or Vercel Edge middleware
- No client-side auth checks as a substitute for server validation
- API responses for admin routes: Cache-Control: no-store

12.7 R3F Performance — Kart Model12.7 R3F Performance — Kart Model

- R3F canvas loads after main content paint — use dynamic import with ssr: false
- Show static fallback image until canvas is ready
- Suspense boundary with fallback so 3D never blocks page render
- On low-end devices / slow connections: detect via navigator.hardwareConcurrency or connection.effectiveType and skip 3D entirely, show static image

```
const KartModelSection = dynamic(() => import('@/components/home/KartModelSection'), {
  ssr: false,
  loading: () => <KartModelPlaceholder />
})
```

12.8 Performance Budget12.8 Performance Budget

Metric	Target	How
First Contentful Paint	< 1.5s on 4G	ISR + CDN serving cached HTML
Largest Contentful Paint	< 2.5s on 4G	next/image lazy loading, WebP thumbnails
Time to Interactive	< 3s on 4G	R3F loaded after main content
Gallery page load	< 2s skeleton → images	Masonry renders skeletons instantly, images lazy load
Admin dashboard	< 3s	SSR acceptable, not user-facing performance-critical

12.9 CI/CD — GitHub Actions12.9 CI/CD — GitHub Actions

Every push to main and every PR runs the following checks. No broken builds merge.

```
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit
      - run: npm run build
```

12.10 Deployment Flexibility12.10 Deployment Flexibility

The codebase must be deployable anywhere without modification:

- Vercel — default, zero config for Next.js
- Self-hosted VPS — via next start or Docker (next.js has official Docker examples)
- College server — same as VPS if Node.js 18+ available

Rules to maintain this: no Vercel-specific APIs (no @vercel/kv, no @vercel/postgres, no edge-only APIs). Use standard Node.js APIs. The only platform dependency is DATABASE_URL and R2 credentials — both are env vars, both are swappable.

12.11 Data Safety12.11 Data Safety

- Neon free tier includes point-in-time restore — no extra config needed
- R2 — never delete objects directly from bucket without removing DB record first
- Admin delete actions: soft-delete preferred (is_active = false) over hard DELETE for team members and sponsors
- Events: never hard delete — set status to 'archived' instead
- Gallery items: hard delete acceptable since photos are not referenced elsewhere

— End of Architecture Document —